

§ PROMETHEUS GRAFANA

1. PROMETHEUS INTRODUCTION

❖ **Before** Prometheus or no monitoring tool

- Before Prometheus and modern tools, monitoring was mostly manual, reactive, and painful

I. Manual Server Checks (Very Old Days)

- Admins used to : SSH into servers, Run commands like : top, free -m, df -h, uptime, ps -ef, netstat -tulnp
- Problems : Only current state, no history, Had to log in after issue happened
 - Not scalable (10 servers ok, 100+ impossible)

II. Log-Based Monitoring : Monitoring = reading logs

- Admins checked : /var/log/syslog, /var/log/messages, Application logs
 - Web server logs (Apache/Nginx)
- Problems : Logs are unstructured, Hard to find root cause, No real-time alerting, No metrics (CPU %, memory trends)

III. Basic Shell Scripts + Cron Jobs :

- People wrote scripts like:

```
if [ $(free | awk '/Mem/ {print $3/$2 * 100}') -gt 80 ]; then
    echo "High Memory Usage" | mail admin@example.com
fi
```
- Problems : Very fragile, Hard to maintain, No visualization, One script per check.

IV. Traditional Monitoring Tools (Pre-Prometheus Era) : Before Prometheus, tools like:

Tool	Type
Nagios	Host/service checks
Zabbix	Agent-based
Cacti	Graphing
Munin	Graph-based
Icinga	Nagios-based

- How they worked : Mostly PUSH-based, Required agents, Config-heavy, Hard to scale for cloud/microservices.

V. Alerts Based on Downtime Only :

- Monitoring meant : “Is server UP or DOWN?”, “Is HTTP 200 or not?”
- No insight into : Latency, Error rates, Resource saturation, Future failures

➔ Life Without Monitoring Tools : If no monitoring at all, teams relied on:

- Users complain first : “Website is slow”, “App is not loading”, “Payment failed”
- This is called : User-reported monitoring (worst kind)
- Result : Reactive firefighting, Long downtime, No SLA/SLO tracking

➔ What Changed With Prometheus

- Prometheus solved modern problems:

Old Way	Prometheus Way
Manual checks	Automated scraping
No history	Time-series data
SSH per server	Centralized view
No alert logic	Smart alert rules
Static servers	Dynamic Kubernetes

- ➔ Why Old Tools Failed for Cloud & Kubernetes : Old tools assumed : Static Ips, Long-living servers
Few services
- Modern world : Containers start/stop, Pods change Ips, Auto-scaling, Microservices
- Prometheus introduced : Service discovery, Label-based metrics, Pull model, Cloud-native monitoring

- “Before Prometheus, monitoring was mostly manual, log-based, or done using traditional tools like Nagios and Zabbix. These methods lacked scalability, real-time metrics, and cloud-native support, making them unsuitable for modern microservices and Kubernetes environments.”

❖ **After Prometheus**

- Prometheus born in age of dynamic infrastructure and grouped up as very effective monitoring system to today’s dynamic infrastructure.
- Monitoring and Alerting on monitoring escalation is very necessary in dynamic, cloud native Micro – Services Infrastructure.
- In absence of a good monitoring system, you have to pay high maintenance and operational cost

2. **PROMETHEUS**

➔ What is Monitoring Tool ?

- A monitoring tool is a software system that continuously observes, collects, and analyzes data from IT systems to ensure they are healthy, available, and performing well.
- Monitoring tool = System health watcher
- A monitoring tool tracks :
 - Infrastructure : CPU usage, Memory usage, Disk space, Network traffic, Server uptime
 - Applications : Response time, Error rate, Request count, Application availability
 - Containers & Cloud : Docker containers, Kubernetes pods & nodes, Cloud VMs (Azure, AWS, GCP)
- Why to Monitor
 - Without monitoring : No idea when or why failures happen, No performance trends, No alerting, Problems are found only after users complain,
 - With monitoring : Detect issues before downtime, Get real-time alerts, Analyze past incidents, Improve system reliability
- Core Functions of a Monitoring Tool : Every monitoring tool does these things:
 - i. Data Collection : Collect metrics, logs, or traces

- ii. Storage : Store data for analysis & history.
- iii. Visualization : Dashboards & graphs
- iv. Alerting : Notify when thresholds are crossed
- v. Analysis : Find root cause & trends

➔ What is Prometheus ?

- Prometheus is an open-source monitoring and alerting tool used to collect, store, and query metrics from applications, servers, containers, and cloud resources.
- It was originally developed by SoundCloud
- Now part of CNCF (Cloud Native Computing Foundation)
- Widely used with Docker, Kubernetes, Azure, AWS

- Prometheus = Metrics Monitoring System

➔ Usage of Prometheus :

- Reason of learning Prometheus is Dynamic infrastructure
- Dynamic infra : Static Infra
 - Virtualization
 - Containers
 - Container Orchestration
 - Cloud computing
 - Cloud computing is dynamic infra
- To monitor that dynamic infra , Prometheus is required.

- Monitor CPU, memory, disk, network
- Monitor applications & microservices
- Monitor Docker containers & Kubernetes pods
- Generate alerts when something goes wrong
- Perform capacity planning
- Troubleshoot performance issues

➔ Prometheus built-in labels Core Concept

i. Target (WHAT Prometheus Scrapes)

- A target is any endpoint that exposes metrics and can be scraped by Prometheus.
- Prometheus pulls metrics from targets
- Usually a URL ending with /metrics
- Example : `http://192.168.1.10:9100/metrics`

ii. Instance (FROM WHERE exactly)

- An instance is a single, unique target address.
- Defined by : `<host>:<port>`
- If two servers run the same service, each one is a different instance
- Example : `instance="192.168.1.10:9100"`

iii. Job (WHO / ROLE of targets)

- A job is a logical group of one or more instances that perform the same function.

- Used for grouping and identification
- Example : `job="node_exporter"`
- Relationship Between Target, Instance, and Job
 - Think of it like this


```

          Job
          └─ Instance
             └─ Target
          
```
- One word :
 - Job → `node_exporter`, Role (Web servers, DB servers)
 - Instances → each IP:PORT, One server/container
 - Targets → actual scrape endpoints, Metrics endpoint
- One Line :
 - Target: "An endpoint exposing metrics for Prometheus to scrape."
 - Instance: "A single target identified by host and port."
 - Job: "A logical group of similar instances."

➔ Prometheus Terminology

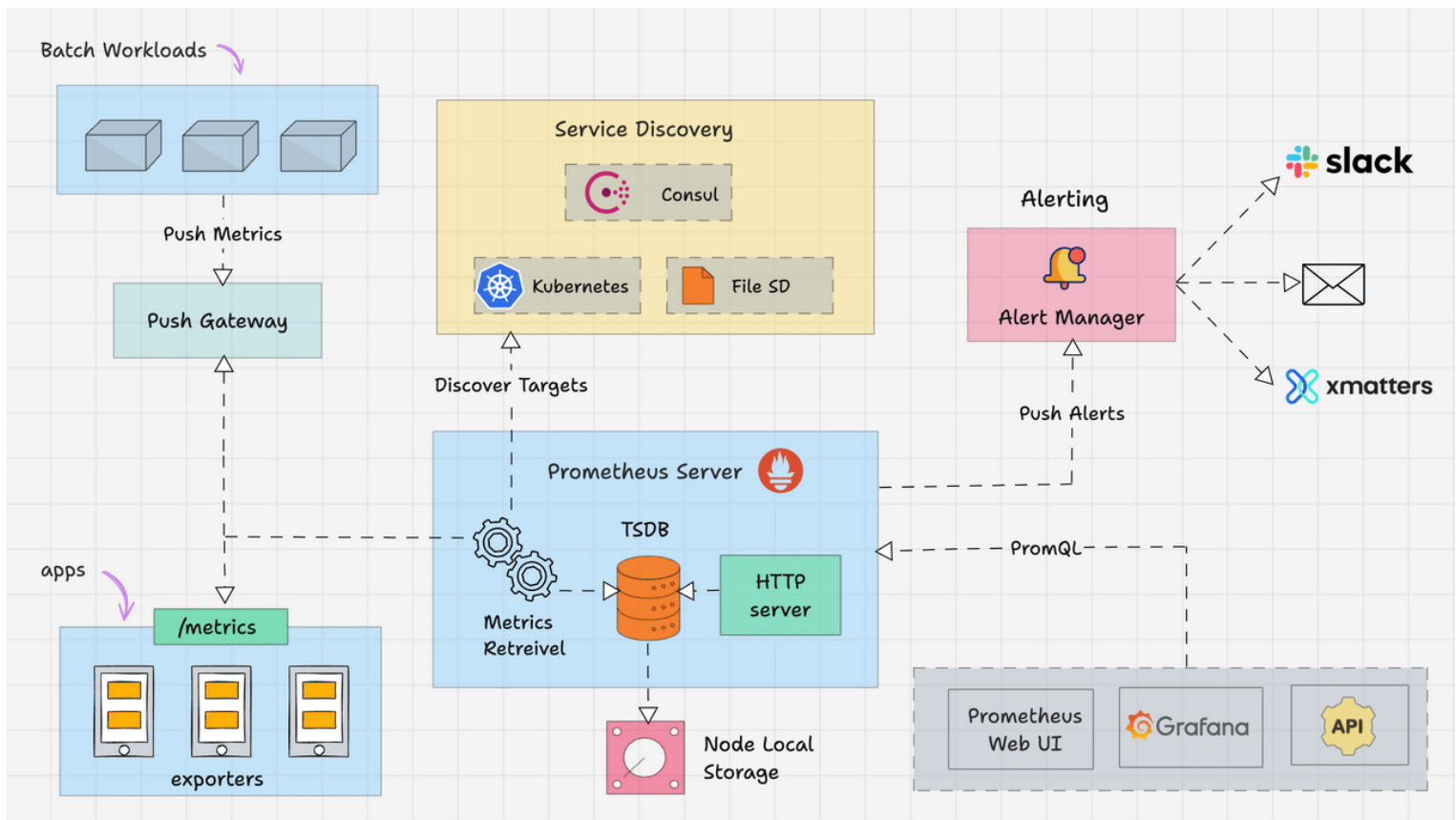
- 1) Monitoring : Monitoring is a systematic process of collecting and recording the activities taking place in a target project, then using that recorded values to check if target reaching their objective or not.
- 2) Alert : It is the outcome of alerting rule in Prometheus
An alert is triggered when a condition is met.
It sent from Prometheus to Alert Manager
Example : CPU > 80% for 5 minutes
- 3) Alert Rule : A rule that defines when an alert fires.
Example : - alert: HighCPUUsage
expr: node_cpu_seconds_total > 80
- 4) Alertmanager : Alert manager handles alerts and sends notifications.
Alert manager takes in alert from Prometheus server.
Do the process on collected data and then send notification as per defined process
Supports : Email, Slack, PagerDuty
- 5) Metric : A metric is a numerical measurement collected over time.
Examples : CPU usage, Memory usage, HTTP request count
- 6) Time Series : A time series is a metric with a timestamp and labels.
Format: `metric_name{label1="value1"} timestamp value`
Example : `node_cpu_seconds_total{cpu="0", mode="idle"}`
- 7) Labels : Labels are key-value pairs attached to metrics to identify them.
Labels make metrics flexible and powerful
Example : `instance="192.168.1.10:9100"`
`job="node_exporter"`

- 8) Target : A target is an endpoint Prometheus scrapes metrics from.
Example : `http://192.168.1.10:9100/metrics`
 - 9) Scraping : Scraping is the process where Prometheus pulls metrics from targets.
Happens at fixed intervals (`scrape_interval`)
 - 10) Job : A job is a group of targets performing the same role.
Example : `job="web_servers"`
 - 11) Exporter : An exporter is a service that exposes metrics in Prometheus format.
Examples : Node Exporter, MySQL Exporter, Blackbox Exporter
 - 12) Sample : A sample is a single data point. (value, timestamp)
 - 13) TSDB (Time Series Database) : Prometheus stores data in a local time-series database.
Optimized for : Fast writes, Fast reads, Time-based queries
 - 14) PromQL : PromQL (Prometheus Query Language) is used to query metrics.
Example : `rate(http_requests_total[5m])`
 - 15) Counter : A counter is a metric that : Only increases, Resets on restart
Example : `http_requests_total`
 - 16) Gauge : A gauge represents a value that : Can increase or decrease
Example : `node_memory_available_bytes`
 - 17) Histogram : Samples observations, Groups them into buckets
Used for : Request duration, Response size
 - 18) Summary : Calculates quantiles (p90, p95), Done on client side
Used for : Latency measurement
 - 19) Service Discovery : Prometheus automatically finds targets.
Examples : Kubernetes, EC2, Azure VMs
 - 20) Recording Rule : Precomputes expensive queries, Stores result as a new metric
Improves performance
 - 21) Instance : An instance is a single target.
Example : `instance="server1:9100"`
- “Prometheus collects metrics as time-series data, identified by labels, scraped from targets, queried using PromQL, and used for monitoring and alerting.”

➔ Components of Prometheus

- Prometheus Architecture has 5 main components :

- I. Prometheus Server : Core component
 - **Retrieval** it will retrieve the metrics from the instance, targets and jobs
 - This will store in the metrics in the storage in the **TSDB – Time Series DB**, to store the custom metrics or metrics Provides PromQL query language
 - This time series DB is accepting the metrics and further will be store on the node, where the Prometheus is running and that is basically store on the Disk, either that is HDD or SSD
 - This Metrics, which time series have, must be retrieved by the **HTTP Server**, either you can use promQL to retrieve the db from the time series or API.
 - Then further HTTP server will send the report to Data Visualization tool or Alert Manager
- II. Exporters / Jobs : Exporters expose metrics in Prometheus format
 - Used when apps don't provide metrics by default
 - These are basically the Prometheus targets, jobs is a combination of multiple instances.
 - Prometheus also supports exporter, exporter use to retrieve the data from windows / linux / Mac os, there are multiple exporters are available.
 - The retrieval from the Prometheus basically pull the metrics from jobs/exporter
 - Note: the jobs and exporter which are running the target, they are not pushing the metric, instead Retrieval component is pulling the metric, **you don't need to install any agent** on the target machine.
 - Examples : Node Exporter → OS metrics (CPU, RAM, Disk), MySQL Exporter, Docker Exporter, Kubernetes Exporter
- III. Pushgateway :
 - Push metrics instead of pull
 - Used for short-lived jobs (batch jobs, cron jobs) OR
 - This component will retrieve from short live jobs like, cron job, scheduler jobs etc.,
 - It is not permanent component ,
 - once retrieval data scrap the data from push gateway, push gateway will be destroyed
 - Example : Backup job, CI/CD job
- IV. Alertmanager :
 - Handles or retrieves alerts sent by Prometheus
 - Prometheus don't have Native tool to push alerts directly to customer.
 - Alert manager will help to push alerts to customers, using Pager duty, email, incident ticket etc.,
 - Sends notifications to : Email, Slack, PagerDuty, OpsGenie
- V. Visualization Tool / Export (Grafana)
 - To view the metrics we will be using this components
 - Prometheus does NOT have strong UI
 - Grafana is used to : Create dashboards, Visualize metrics
- VI. Service Discovery :
 - There are multiple instances, like Kubernetes etc., it may scale up and scale down.
 - This component will discover the instances which scale up and scale down.
 - Again retrieval will retrieve the data which is discover.



3. PROMETHEUS ARCHITECTURE

➔ Prometheus is an open-source monitoring and alerting tool used to collect and store metrics (CPU, memory, requests, errors, etc.) from applications and servers.

- It mainly works on a Pull Model.

➔ Prometheus Architecture – Main Components

I. Prometheus Server

- Central monitoring engine
- Scrapes metrics from targets using HTTP (pull model)
- Stores data in Time Series Database (TSDB)
- Executes PromQL queries
- Evaluates alert rules

II. Targets

- Applications, servers, containers, or services being monitored

- Expose metrics via /metrics HTTP endpoint
- Provide metrics in Prometheus text format

III. Exporters

- Agents that collect system or third-party metrics
- Convert metrics into Prometheus format
- Example: Node Exporter (server metrics), DB Exporter

IV. TSDB (Time Series Database)

- Built-in storage inside Prometheus
- Stores metrics as time-stamped key-value pairs
- Uses WAL (Write-Ahead Log) and data blocks

V. Alertmanager

- Handles alerts generated by Prometheus
- Groups, silences, and routes alerts
- Sends notifications (Email, Slack, etc.)

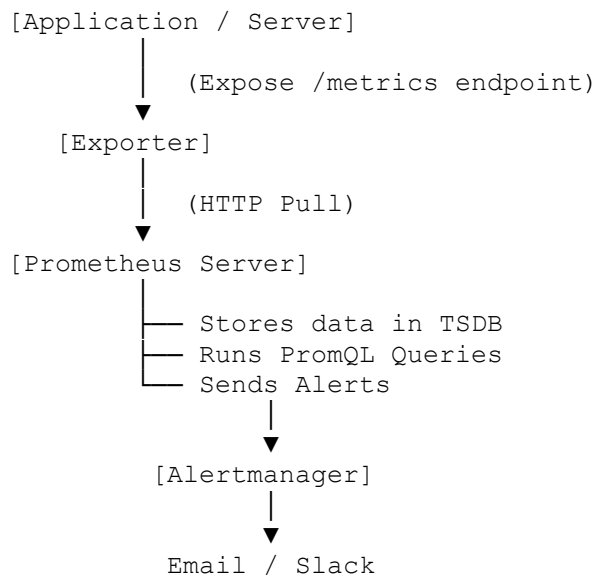
VI. Service Discovery

- Automatically discovers dynamic targets
- Common in cloud and Kubernetes environments

VII. Grafana (Visualization Tool)

- Used to create dashboards
- Connects to Prometheus as data source

➔ Prometheus Workflow (Step-by-Step)



➔ Simple :

Prometheus scrapes metrics from targets, stores them in a time-series database, evaluates rules, and sends alerts via Alertmanager while Grafana visualizes the data.

4. PROMETHEUS INTERNAL ARCHITECTURE

➔ Prometheus internally scrapes metrics, writes them to WAL, stores recent data in memory (Head), persists them as time-series blocks on disk, and processes queries and alert rules using its built-in query engine.

- Components :

- I. Scrape Manager :

- Manages all configured targets
- Sends HTTP requests to /metrics endpoints
- Collects metrics at defined scrape intervals
- Works on Pull Model

- II. Service Discovery Manager

- Dynamically discovers targets
- Supports cloud & Kubernetes environments
- Updates target list automatically

- III. TSDB (Time Series Database Engine)

- Prometheus has a built-in TSDB.
- It stores data as : `metric_name{label="value"} timestamp value`
- Example : `cpu_usage{instance="server1"} 1709000000 75`
- Each metric = Time Series

- IV. WAL (Write-Ahead Log)

- First writes incoming data to WAL
- Prevents data loss during crash
- Helps in recovery
- Data Flow : Scraped Data → WAL → Memory → Disk Blocks

- V. Head Block (In-Memory Storage)

- Recent data stored in memory
- Fast read & write
- Later compacted into persistent blocks

- VI. Block Storage (Persistent Storage)

- Data stored in 2-hour blocks (default)
- Each block contains : Chunks (actual metric data)
 - Index (for fast search)
 - Metadata

- VII. Compaction Process

- Merges small blocks into larger blocks

- Improves storage efficiency
- Reduces disk usage

VIII. Query Engine

- Executes PromQL queries
- Reads data from : Head block, Persistent blocks
- Returns results via HTTP API

IX. Rule Engine

- Evaluates : Recording Rules, Alerting Rules
- Runs at configured intervals

X. HTTP Server

- Provides : Web UI
- API access
- /metrics endpoint (self-metrics)

➔ Internal Data Workflow

Target → Scrape Manager

↓

WAL

↓

Head (Memory)

↓

Persistent Blocks

↓

Query Engine

↓

Web UI / API / Alerts

5. PROMETHEUS MONITORING TOOL PRACTICAL

➔ Prometheus monitoring tool practical step by step

Commands :

Optional part for creating directory/ folder and for downloading but it gives clarity for avoiding confusion

Step1 : For creating a directory : mkdir downloads/

Step2 : For moving to directory : cd downloads/

Mandatory from here

Step3 : For getting link : wget

<https://github.com/prometheus/prometheus/releases/download/v3.10.0/prometheus-3.10.0.linux-amd64.tar.gz>

Check : Click on link working or not

If working well and good else replace link with latest version

Step4 : For coming back one folder (if downloads created): cd ..

Step5 : For extracting : tar -xvf ./downloads/prometheus-3.10.0.linux-amd64.tar.gz

Press tab for full name automatic

Step6 : For listing : ls

Step7 : For changing name : mv prometheus-3.10.0.linux-amd64/ prometheus

Step8 : For moving to Prometheus directory : cd prometheus/

Step9 : For listing : ls

Step 10 : For executing Prometheus 1: ./prometheus

For executing Prometheus in background 2 : ./prometheus &

For executing prometheus background 3 : nohup ./prometheus > prometheus.log2>&1 &

1 executes but if exited process stops

2 and 3 executes in background so we can exit safe

Now in text it shows starting server

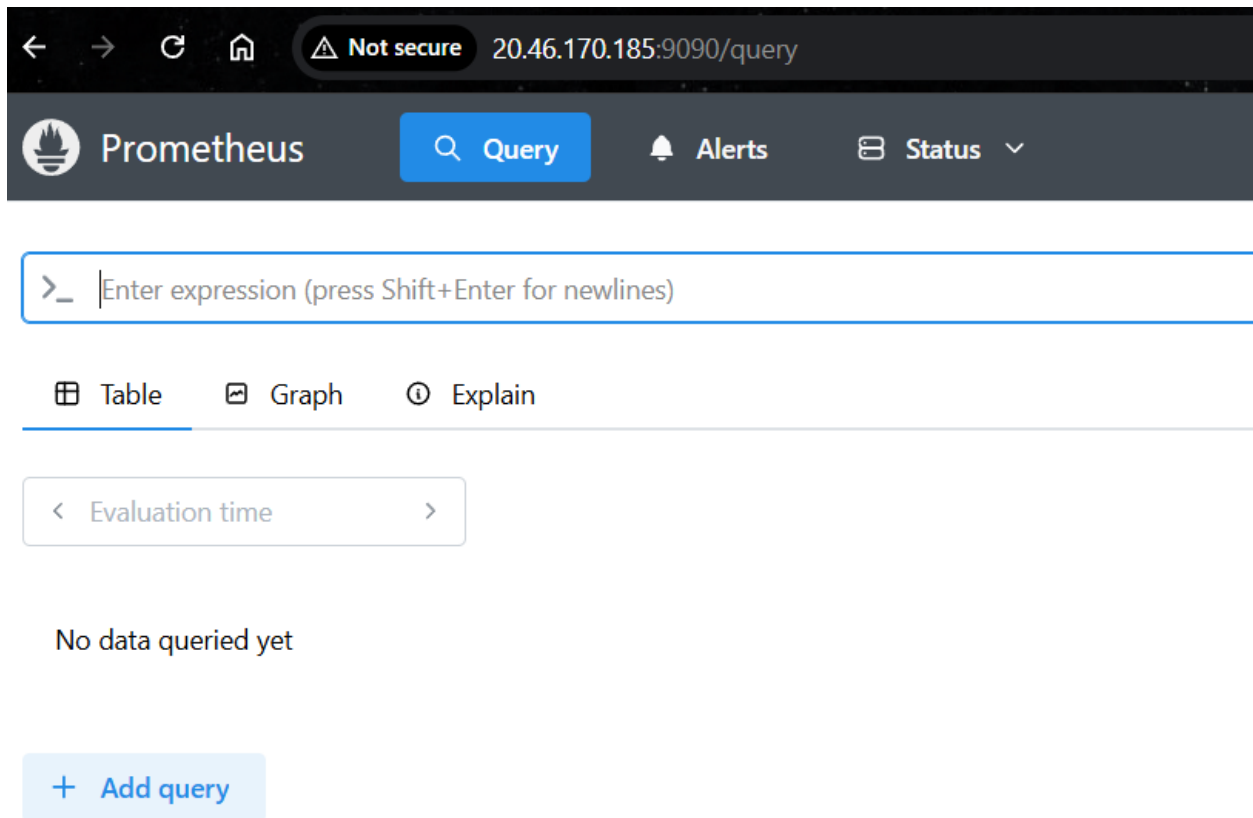
Starting tsdb

Starting rule manager

Go language

Port 9090

For web view : <ipaddress>:9090



For checking Prometheus details and go language version

We can select option beside Prometheus select status and runtime & build information

Version : 3.10.0

Go Version : 1.26.0

Step 11 used when you use 1 and 2 : For stoping : ctrl + c

Step 12 : For checking port : pgrep Prometheus

You will get process ID

Step 13 : For stoping prometheus completely : kill <processID>

Done ok else use : kill -9 <processID>

Not secure 20.46.170.185:9090/status

Prometheus Query Alerts Status > Runtime & build information

Build information

Version	3.10.0
Revision	54e010926b0a270cadb22be1113ad45fe9bcb90a
Branch	HEAD
BuildUser	root@2ee2be8e6652
BuildDate	20260225-21:12:02
GoVersion	go1.26.0

Runtime information

Start Time	2026-02-27T06:16:02Z
Working Directory	/home/SaiAadmin/prometheus
Hostname	SaiD
Server Time	2026-02-27T06:24:02Z
Configuration Reload	Successful
Last Successful Configuration Reload	2026-02-27T06:16:02Z
WAL Corruptions	0

6. PROMETHEUS NODE EXPORTER

- ❖ **Prometheus Node Exporter** is an exporter that collects hardware and OS-level metrics from a Linux/Unix system and exposes them in Prometheus format.
 - It does NOT monitor applications
 - It monitors the host (server / VM / node)
- ➔ **Why Node Exporter Is Used :** Node Exporter is used to monitor server health and performance.
 - It provides visibility into : CPU usage, Memory usage, Disk usage, Network traffic, Load average, File system stats, Hardware info
 - Without Node Exporter : Prometheus cannot see OS-level metrics
- ➔ **What Metrics Does Node Exporter Provide :**
 - Some common metrics are :-
 - 🖥️ CPU : node_cpu_seconds_total
 - 🧠 Memory : node_memory_MemAvailable_bytes
 - 💾 Disk : node_filesystem_avail_bytes
 - 🌐 Network : node_network_receive_bytes_total
 - ⚙️ System Load : node_load1
node_load5
node_load15

➔ How Node Exporter Works :-

- Runs as a service or container on the server
- Collects metrics from : /proc, /sys
- Exposes metrics on : <http://<node-ip>:9100/metrics>
- Prometheus scrapes this endpoint

➔ Simple Architecture

Server / VM

└─ Node Exporter (port 9100)

└─ /metrics

↑

Prometheus Server (pull)

➔ Node Exporter vs Application Monitoring

Node Exporter	App Metrics
OS-level	App-level
CPU, RAM, Disk	Requests, latency
One per node	One per app

- ➔ “Prometheus Node Exporter is an agent that runs on a host to expose system-level metrics such as CPU, memory, disk, and network usage for monitoring.”

7. PROMETHEUS NODE EXPORTER

- ➔ Node Exporter is an open-source metrics exporter for Linux/Unix systems that collects system-level hardware and operating system metrics and exposes them through an HTTP endpoint (/metrics) so monitoring systems like Prometheus can scrape them.

- Node Exporter is a Prometheus exporter that gathers host-level metrics from the Linux kernel and system resources and exposes them in Prometheus format for monitoring.

- Gather all the data and transform into Prometheus format
- Prometheus server will retrieve data
- Retrieval server will scrap the data and it will pull the data to Prometheus server
- Time series database will store
- http end points are used to trigger data and web ui will be used to get data

- ➔ Uses of Node Exporter : Node Exporter is used for infrastructure monitoring.

- Main Reasons
 - I. System Monitoring : Collects CPU, memory, disk, and network statistics.
 - II. Prometheus Integration : Provides metrics in a format that Prometheus can scrape.
 - III. Infrastructure Observability : Helps DevOps teams monitor servers, VMs, and cloud instances.
 - IV. Visualization in Dashboards : Metrics can be visualized using Grafana dashboards.
 - V. Alerting : Metrics can trigger alerts through Prometheus and Alertmanager.

➔ How Node Exporter Works

Server (Linux Machine)



Node Exporter (collects system metrics)



Prometheus (scrapes /metrics endpoint)



Grafana (visualizes dashboards)

- Interview point :
Node Exporter is a Prometheus exporter that collects system-level metrics such as CPU, memory, disk, and network usage from a host and exposes them via an HTTP endpoint for Prometheus to scrape and monitor infrastructure performance.

8. PROMETHEUS NODE EXPORTER PRACTICAL

➔ Steps for installing

- Download node exporter on target machine
- Extract tar file and start node exporter
- Check if publishing Node exporter will work on 9100
- What monitor we need, depends on monitor requirement we use monitor node exporter
- We can check and download node exporter from :
<https://prometheus.io/docs/instrumenting/exporters/>
- Depends on node we need

Open Prometheus.io website

Downloads -> for os

Optional part (Create a folder downloads

Use command : wget and download Prometheus using web link)

Step1 : Create folder or directory : mkdir downloads

Step2 : Move to downloads : cd downloads/

Step3 : For downloading : wget

https://github.com/prometheus/node_exporter/releases/download/v1.10.2/node_exporter-1.10.2.linux-amd64.tar.gz

Check this link opening or not

If working ok else replace it with new link

Step4 : For moving from downloads : `cd ..`
Step5 : For extracting node exporter : `tar -xvf ./downloads/node_exporter-1.10.2.linux-amd64.tar.gz`

Step6 : Change folder name to node_exporter : mv node_exporter-1.10.2.linux-amd64/ node_exporter

Step 7 : Change folder to node exporter : cd node_exporter/

Step8 : For execution 1: ./node_exporter

For execution in background : ./node_exporter &

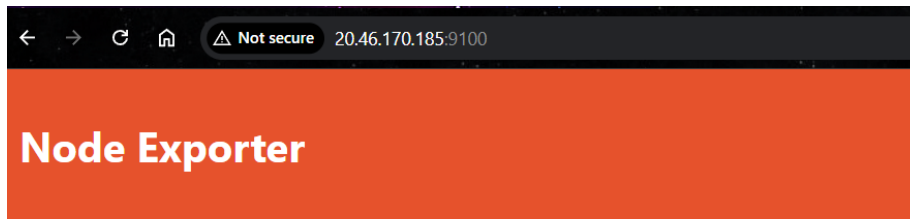
For execution in background : nohup ./node_exporter > node_exporter.log2>&1 &

For 1 we need to type ctrl + d for exiting but it stops

For 2 and 3 we need to type ctrl + d for exiting but it keeps running

Step 9 : For checking node exporter working or not

Step10 : Open web browser and type : <vm Pip>:9100



Prometheus Node Exporter

Version: (version=1.10.2, branch=HEAD, revision=654f19dee6a0c41de78a8d6d870e8c742cdb43b9)

- [Metrics](#)

Download a detailed report of resource usage (pprof format, from the Go runtime):

- [heap usage \(memory\)](#)
- [CPU usage \(60 second profile\)](#)

To visualize and share profiles you can upload to [pprof.me](#)

By executing node exporter we get in this way :

```
time=2026-02-28T06:09:54.608Z level=INFO source=node_exporter.go:141 msg=xfs
time=2026-02-28T06:09:54.608Z level=INFO source=node_exporter.go:141 msg=zfs
time=2026-02-28T06:09:54.608Z level=INFO source=tls_config.go:346 msg="Listening on" address=[::]:9100
time=2026-02-28T06:09:54.608Z level=INFO source=tls_config.go:349 msg="TLS is disabled." http2=false address=[::]:9100
```

Now we need to write job here

For monitoring

Step 10 : Move back : `cd ..`

Step 11 : Move to prometheus folder : `cd prometheus/`

Step 12 : Now enter yaml file : `vi prometheus.yml`

Check this block :

- job_name: "prometheus"

metrics_path defaults to '/metrics'

scheme defaults to 'http'.

static_configs:

- targets: ["localhost:9090"]

The label name is added as a label `label\_name=<label\_value>` to any timeseries scraped from this config.

labels:

app: "prometheus"

- Copy paste it again and edit as : change job name and app name as
 - prometheus to node_exporter
 - 9090 to 9100

```

scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]
        # The label name is added as a label `label_name=<label_value>` to any timeseries scraped from this config.
        labels:
          app: "prometheus"

  - job_name: "node_exporter"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9100"]
        # The label name is added as a label `label_name=<label_value>` to any timeseries scraped from this config.
        labels:
          app: "node_exporter"

```

Now restart prometheus server

Check prometheus server :

Step13 : For checking : pgrep prometheus

Step 14 : For stoping : kill <service number>

Step 15 : For restarting : ./prometheus &

Check in web : <ip address>:9090

Now check status : health

Target Group	Endpoint	Labels	Last scrape	State
node_exporter	http://localhost:9100/metrics	app="node_exporter" instance="localhost:9100" job="node_exporter"	4.022s ago 9ms	UP
prometheus	http://localhost:9090/metrics	app="prometheus" instance="localhost:9090" job="prometheus"	7.89s ago 2ms	UP

For node memory query :

Query	Result
node_memory_MemFree_bytes	2876239872

We can add and search multiple queries

The screenshot shows the Prometheus web interface. The top navigation bar includes 'Prometheus', 'Query', 'Alerts', and 'Status'. Below the navigation bar, there are two query input sections. The first section has a query input field containing 'node_memory_MemFree_bytes', a 'Table' view selector, and an 'Execute' button. Below the query input, there is a 'Load time: 223ms Result series: 1' indicator and a table showing the result of the query: 'node_memory_MemFree_bytes(app="node_exporter", instance="localhost:9100", job="node_exporter")' with a value of '2876239872'. The second section has a query input field containing 'node_disk_read_bytes_total', a 'Table' view selector, and an 'Execute' button. Below the query input, there is a 'Load time: 204ms Result series: 1' indicator and a table showing the result of the query: 'node_disk_read_bytes_total(app="node_exporter", device="nvme0n1", instance="localhost:9100", job="node_exporter")' with a value of '757917696'. At the bottom of the interface, there is a '+ Add query' button.

9. PROMETHEUS ALERT MANAGER

➔ Alert manager is a centralized alert processing system that manages, groups, routes, and sends alerts generated by Prometheus monitoring rules.

Or

➔ Alertmanager is a Prometheus component responsible for handling alerts generated by Prometheus.

- It receives alerts, processes them, and sends notifications to configured receivers such as email, Slack, or PagerDuty.

➔ How Alert manager Works

- Node Exporter → Prometheus → Alertmanager → Notification (Email / Slack / etc.)

- Metrics are collected from exporters.
- Prometheus evaluates alert rules.
- When a condition is met, Prometheus fires an alert.
- The alert is sent to Alertmanager.
- Alertmanager processes and sends notifications.

➔ Why Alertmanager is Important

- I. Alert Routing : Alertmanager sends alerts to different teams or channels based on labels.
- II. Alert Grouping : Multiple alerts are combined into one notification to reduce alert noise.
- III. Deduplication : Prevents sending duplicate alerts repeatedly.
- IV. Silencing Alerts : Temporarily mute alerts during maintenance windows.

- V. Alert Inhibition : Suppresses lower-priority alerts when a higher-priority alert exists.

Main Functions of Alertmanager

Feature	Description
Grouping	Combines multiple similar alerts into one notification
Deduplication	Prevents duplicate alerts
Routing	Sends alerts to different receivers based on labels
Silencing	Temporarily suppresses alerts
Inhibition	Stops lower-priority alerts when a higher-priority alert is active

- Interview point : Alertmanager is a Prometheus component that handles alerts by performing alert grouping, deduplication, routing, silencing, and sending notifications to external systems such as email or Slack.

10. PROMETHEUS ALERT MANAGER PRACTICAL

- Prometheus alerting rules
- Alerting rules define your alerting conditions,
- Based on conditions it will send alert or inform
- If alert trigger it will send alert manager, it manager will send notify

In yaml files we write jobs to scan and get status

For getting alert we write rules

In yaml we write rules, using this yaml it will send alert to alert manager it will send to outside

Step 1 : In prometheus.yml file (remove # symbol to activate it)

rule_files:

- "first_rules.yml"
- "second_rules.yml"

Step2 : In first_alert.yml file

Sample one with simple : whether system is up or running or not

groups:

- name: node_alert
rules:
alert: node_exporter_status
expr: up{job="node_exporter"} == 0

Sample rule 2 with extended mentioned present

groups:

- name: node_alert
rules:
Node Exporter Status
- alert: node_exporter_status
expr: up{job="node_exporter"} == 0

```
for: 1m
labels:
  severity: critical
annotations:
  summary: "Node Exporter Down"
  description: "Node exporter is not running on {{ $labels.instance }}"
```

High CPU Usage

```
- alert: high_cpu_usage
  expr: 100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High CPU Usage"
    description: "CPU usage is above 80% on {{ $labels.instance }}"
```

High Memory Usage

```
- alert: high_memory_usage
  expr: (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100 > 80
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "High Memory Usage"
    description: "Memory usage is above 80% on {{ $labels.instance }}"
```

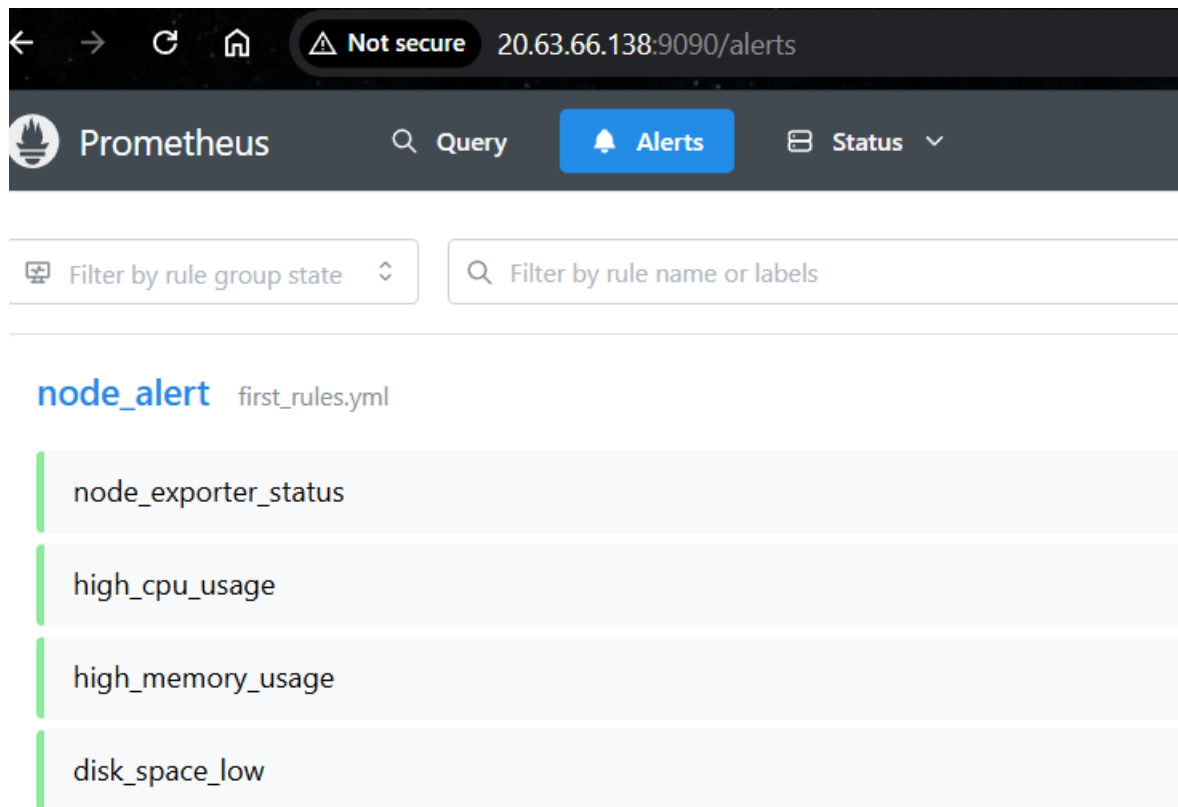
Low Disk Space

```
- alert: disk_space_low
  expr: (node_filesystem_avail_bytes{mountpoint="/"}/
node_filesystem_size_bytes{mountpoint="/"}) * 100 < 20
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "Low Disk Space"
    description: "Disk space less than 20% available on {{ $labels.instance }}"
```

Step3 : After writing rules

Restart prometheus and node server

Observe this in alerts



Step4 : in prometheus.yml file (we can change time interval based on we need)

my global config

global:

 scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.

 evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.

scrape_timeout is set to the global default (10s). #Alertmanager configuration

alerting:

 alertmanagers:

 static_configs:

 targets:

 # alertmanager: 9093 .

#Load rules once and periodically evaluate them according to the global 'evaluation_interval'.

rule files:

 "first_rules.yml"

 "second_rules.yml"

A scrape configuration containing exactly one endpoint to scrape:

#Here it's Prometheus itself.

Step 5 : to check rules working correctly or not

Step6 : To check promtool version : promtool --version

Step 7 : To check working or not : ./promtool check rules first_rules.yml

```
root@Dockersai:/home/SaiAzadmin/prometheusapp# cat first_rules.yml
groups:
- name: node_alert
  rules:
  - alert: node_exporter_status
    expr: up{job="node_exporter"} == 0

root@Dockersai:/home/SaiAzadmin/prometheusapp# ./promtool check rules first_rules.yml
Checking first_rules.yml
SUCCESS: 1 rules found
```

➔ Now for alert manager

Edit prometheus.yml file : vi prometheus.yml

Check alert manager name :

From : # - alertmanager:9093

To as it is local : - localhost:9093

```
# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093
```

Modify it with

```
# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - localhost:9093
```

Prometheus alert manager website : <https://prometheus.io/download/#alertmanager>

For installing alert manager

Step 8 : Be in downloads folder : cd downloads/

Step 9 : For downloading file : wget

<https://github.com/prometheus/alertmanager/releases/download/v0.31.1/alertmanager-0.31.1.linux-amd64.tar.gz>

Check on link is working or not

If no replace with latest link

Step 10 : For extracting file : tar -xzf ./downloads/alertmanager-0.31.1.linux-amd64.tar.gz

Step 11 : For changing name : mv alertmanager-0.31.1.linux-amd64 alertmanager

Step 12 : For moving to alert folder : cd alertmanager/

Step 13 : restart prometheus, node exporter and alert manager

Step 14 : check service number : `pgrep <service name>`

Prometheus/node_exporter/alertmanager

Step 15 : Stop the services by killing it : `kill <service number>`

Again check : `pgrep <service number>`

If not found well and good

If found force stop it using : `kill -9 <service number>`

Step 16 : go to folder and start services

Go to prometheus folder from home : `cd prometheus/`

Start prometheus : `./prometheus &`

Go to node exporter folder : `cd node_exporter/`

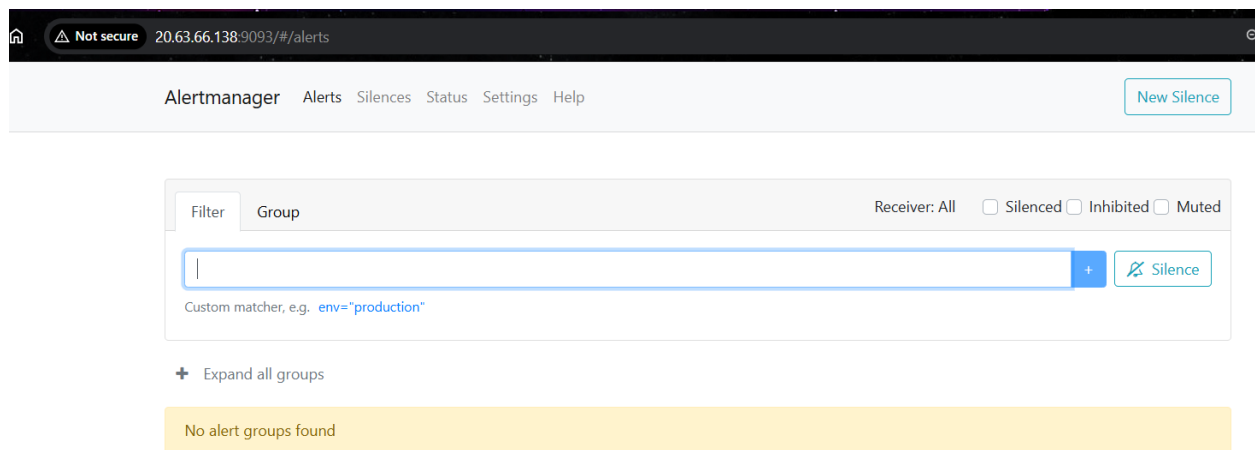
Start node exporter : `./node_exporter &`

Go to alert manager folder from home : `cd alertmanager :`

Start alert manager : `nohup ./alertmanager > alertmanager.log 2>&1 &`

For checking service number type : `pgrep alertmanager`

Now access in web using ip address and port access alert manager : `<ipaddress>:9093`



11. GRAFANA VISUALIZATION TOOL

➔ Grafana is an open-source monitoring and visualization platform used to query, analyze, and display metrics from different data sources in dashboards.

Or

- Grafana is a data visualization and observability tool that connects to monitoring systems like Prometheus, queries their metrics, and displays them as graphs, charts, and dashboards.

➔ How Grafana Works

- Node Exporter → Prometheus → Grafana Dashboard

➔ Exporters collect system metrics (CPU, memory, disk).

- Prometheus stores these metrics.
- Grafana queries Prometheus.
- Grafana displays metrics as visual dashboards.

➔ Main Features of Grafana

Feature	Description
Dashboards	Visual panels for monitoring metrics
Data Source Integration	Connects to Prometheus, Elasticsearch, MySQL, etc.
Visualization	Graphs, gauges, tables, heatmaps
Alerting	Can trigger alerts based on thresholds
Real-time Monitoring	Displays live infrastructure metrics

- Interview point :
Grafana is an open-source visualization and monitoring tool used to create dashboards and analyze metrics collected from data sources like Prometheus.

12. GRAFANA VISUALIZATION TOOL PRACTICAL

Official Grafana website : Grafana.com

Download from : <https://grafana.com/grafana/download?pg=get&plcmt=selfmanaged-box1-cta1>

Choose version from it

Select edition as OSS not enterprise

Installing method 1 :

```
sudo apt-get install -y adduser libfontconfig1 musl
wget https://dl.grafana.com/grafana/release/12.4.0/grafana_12.4.0_22325204712_linux_amd64.deb
sudo dpkg -i grafana_12.4.0_22325204712_linux_amd64.deb
```

Installing method 2 :

Add Grafana repository

```
sudo apt-get install -y apt-transport-https software-properties-common wget
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee /etc/apt/sources.list.d/grafa
```

Update packages : sudo apt update

Install Grafana : sudo apt install grafana -y

Steps to start Grafana :

Step 1 : Start Grafana Service : sudo systemctl start grafana-server

Step 2 : Enable auto start : sudo systemctl enable grafana-server

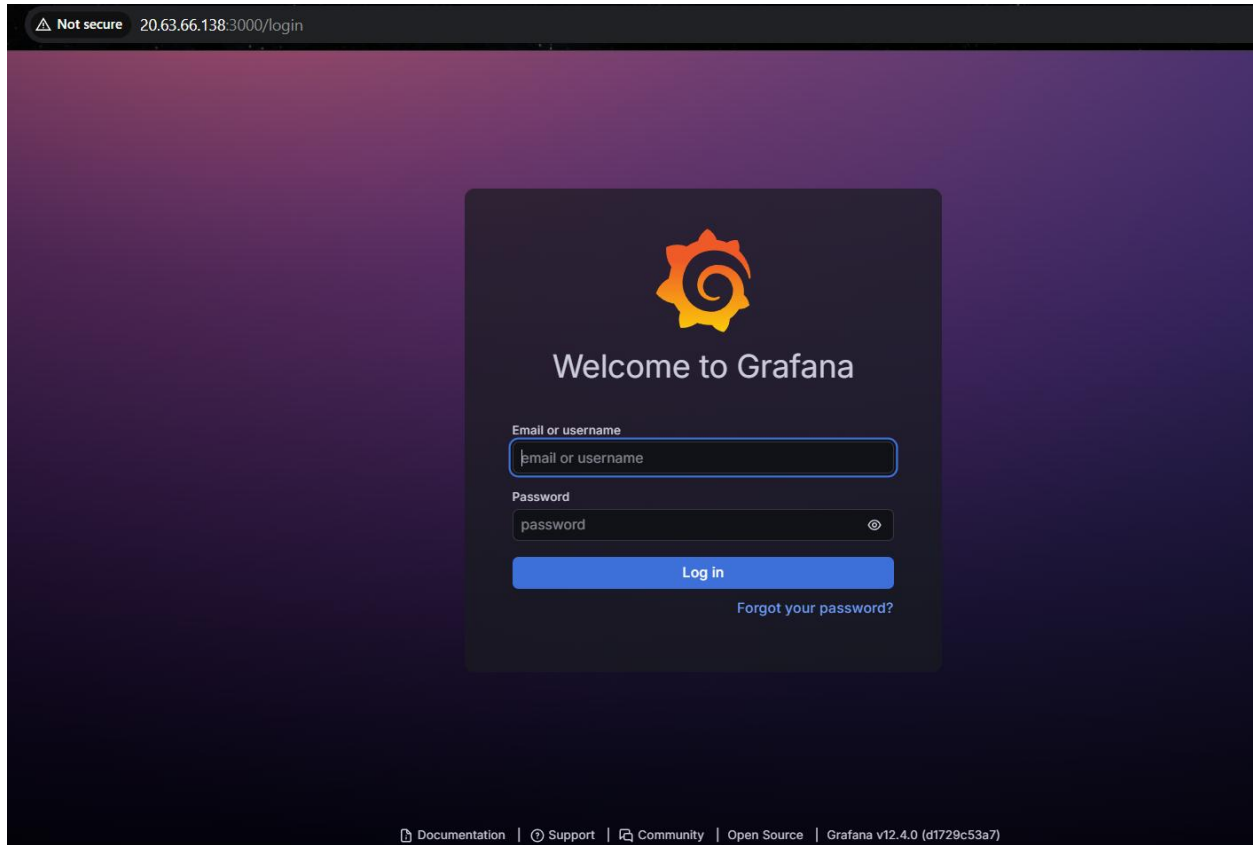
Check status : sudo systemctl status grafana-server

Grafana runs on **port 3000 by default**.

It asks username and password

Username = admin

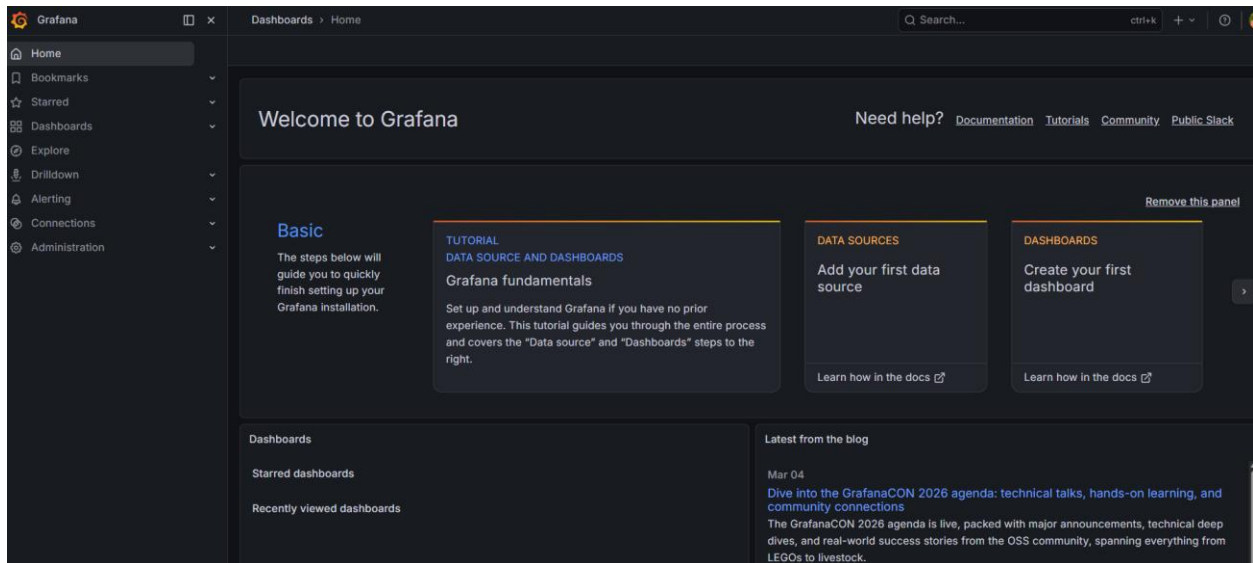
Password = admin



The image shows the Grafana login page. At the top, a browser address bar displays '20.63.66.138:3000/login' with a 'Not secure' warning. The page has a dark purple gradient background. In the center, there is a dark grey login box. At the top of this box is the Grafana logo (a yellow and orange gear-like shape). Below the logo, the text 'Welcome to Grafana' is displayed. Underneath, there are two input fields: 'Email or username' and 'Password'. The 'Email or username' field contains the placeholder text 'email or username'. The 'Password' field contains the placeholder text 'password' and has an eye icon to its right. Below these fields is a blue 'Log in' button. To the right of the button is a link that says 'Forgot your password?'. At the bottom of the page, there is a footer with links for 'Documentation', 'Support', 'Community', 'Open Source', and 'Grafana v12.4.0 (d1729c53a7)'.

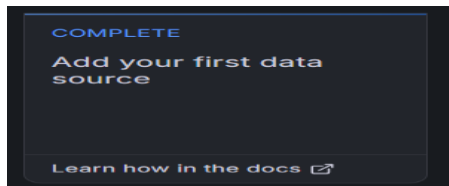
Change password you like else leave it

After login



Left bar

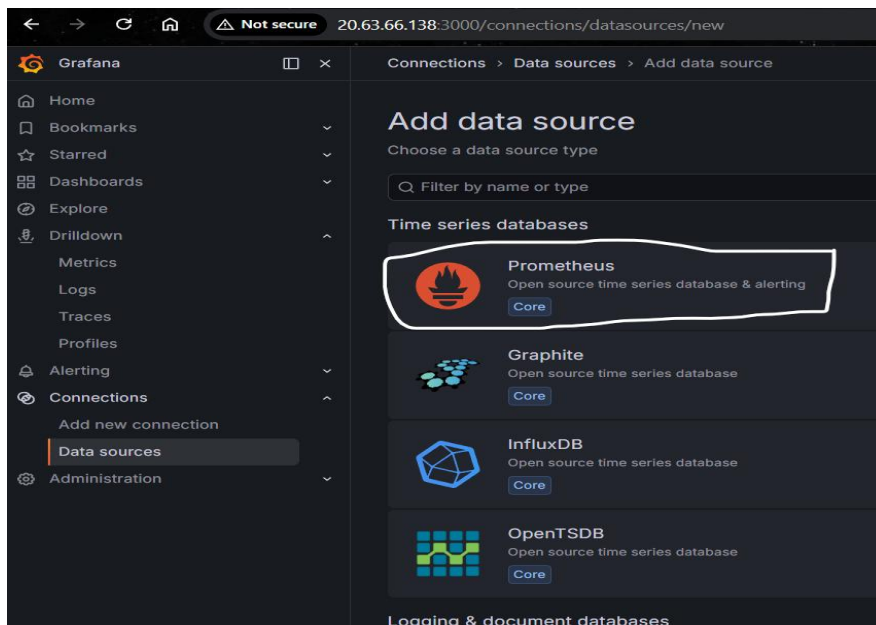
In left side you can find home click on it



Check add your data source and click on it

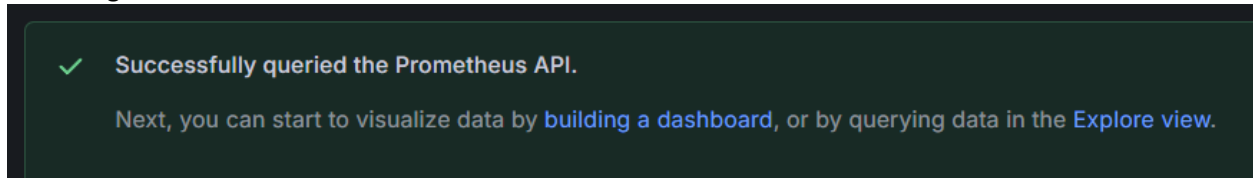
In add data source you can find many tools as Grafana supports many tools

Click on prometheus



Enter prometheus server ip address with port attach/
Ex : <http://20.63.66.138:9090/>

You will get this

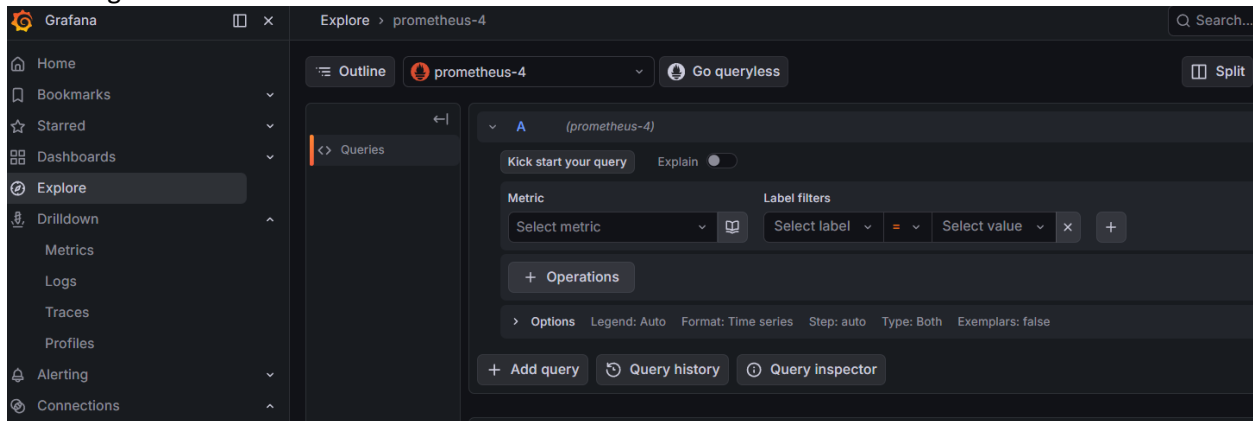


If error occurs rectify it

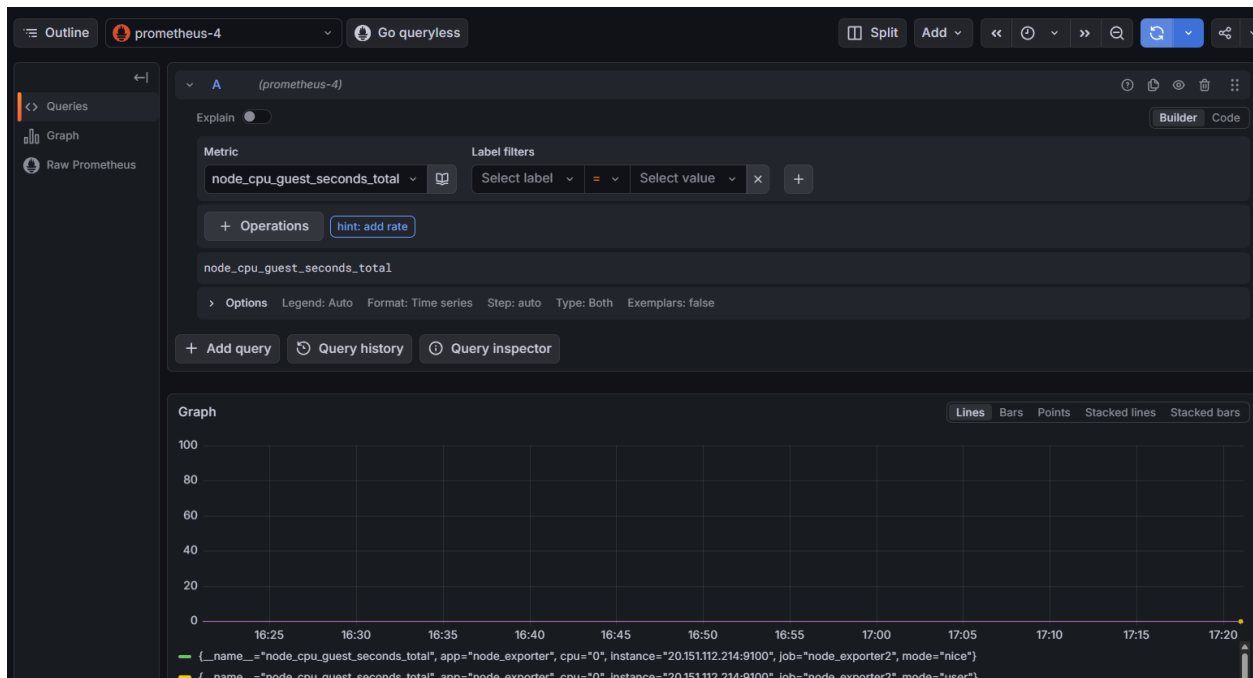
- After that

Click on explore view

You can get this view



In metric type any metric name and click on run query
Else click on rotate



Raw Table Raw

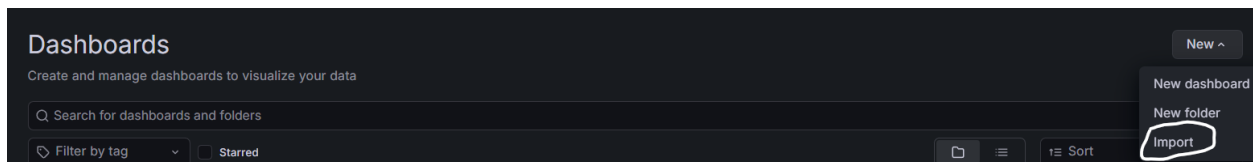
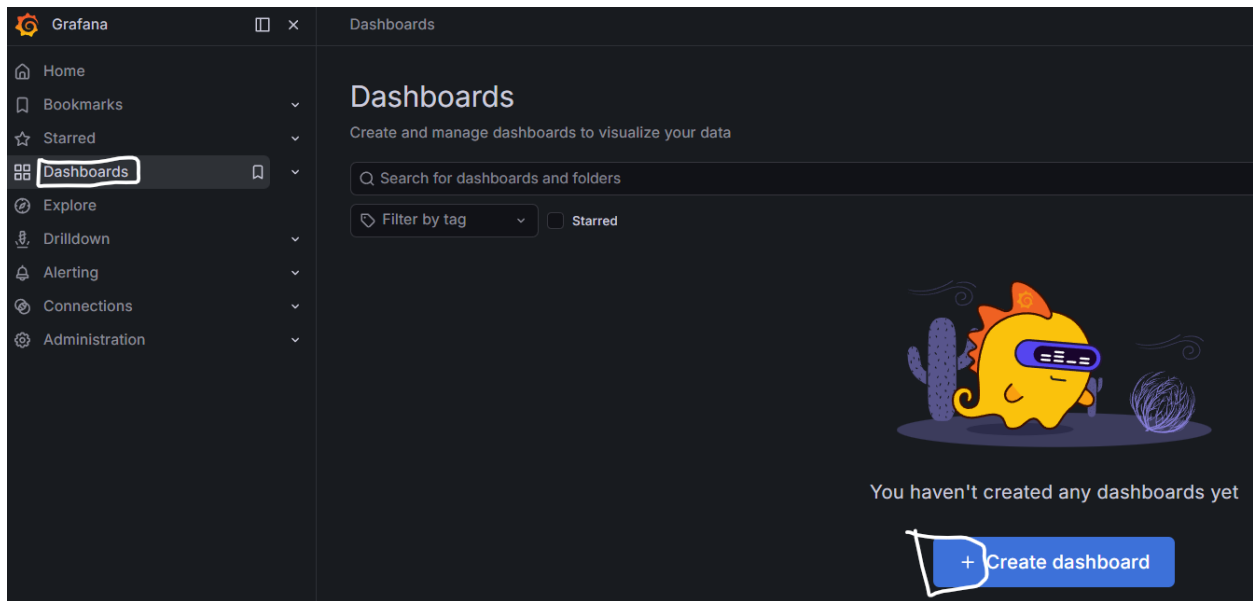
Expand results

Result series: 8

node_cpu_guest_seconds_total{app="node_exporter", cpu="0", instance="20.151.112.214:9100", job="node_exporter2", mode="nice"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="0", instance="20.151.112.214:9100", job="node_exporter2", mode="user"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="1", instance="20.151.112.214:9100", job="node_exporter2", mode="nice"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="1", instance="20.151.112.214:9100", job="node_exporter2", mode="user"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="0", instance="localhost:9100", job="node_exporter1", mode="nice"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="0", instance="localhost:9100", job="node_exporter1", mode="user"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="1", instance="localhost:9100", job="node_exporter1", mode="nice"}	0
node_cpu_guest_seconds_total{app="node_exporter", cpu="1", instance="localhost:9100", job="node_exporter1", mode="user"}	0

We can put multiple queries

➔ Now we need dashboard



From Grafana server dashboard link : <https://grafana.com/grafana/dashboards/>

Search any dash boards related to Prometheus → Node Exporter → Grafana

Import dashboard

Import dashboard from file or Grafana.com



Upload dashboard JSON file

Drag and drop here or click to browse

Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

Grafana.com dashboard URL or ID

Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_0HnEoN4z",
  "panels": [...]
  ...
}
```

Load

Cancel

Click on load

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](https://grafana.com)

Published by

StarsL.cn

Updated on

2023-07-07 13:10:39

Options

Name

Node Exporter Dashboard EN 20201010-StarsL.cn

Folder

📁 Dashboards

Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

xfpJB9FGz

Change uid

DS_VICTORIAMETRICS

Select a prometheus data source

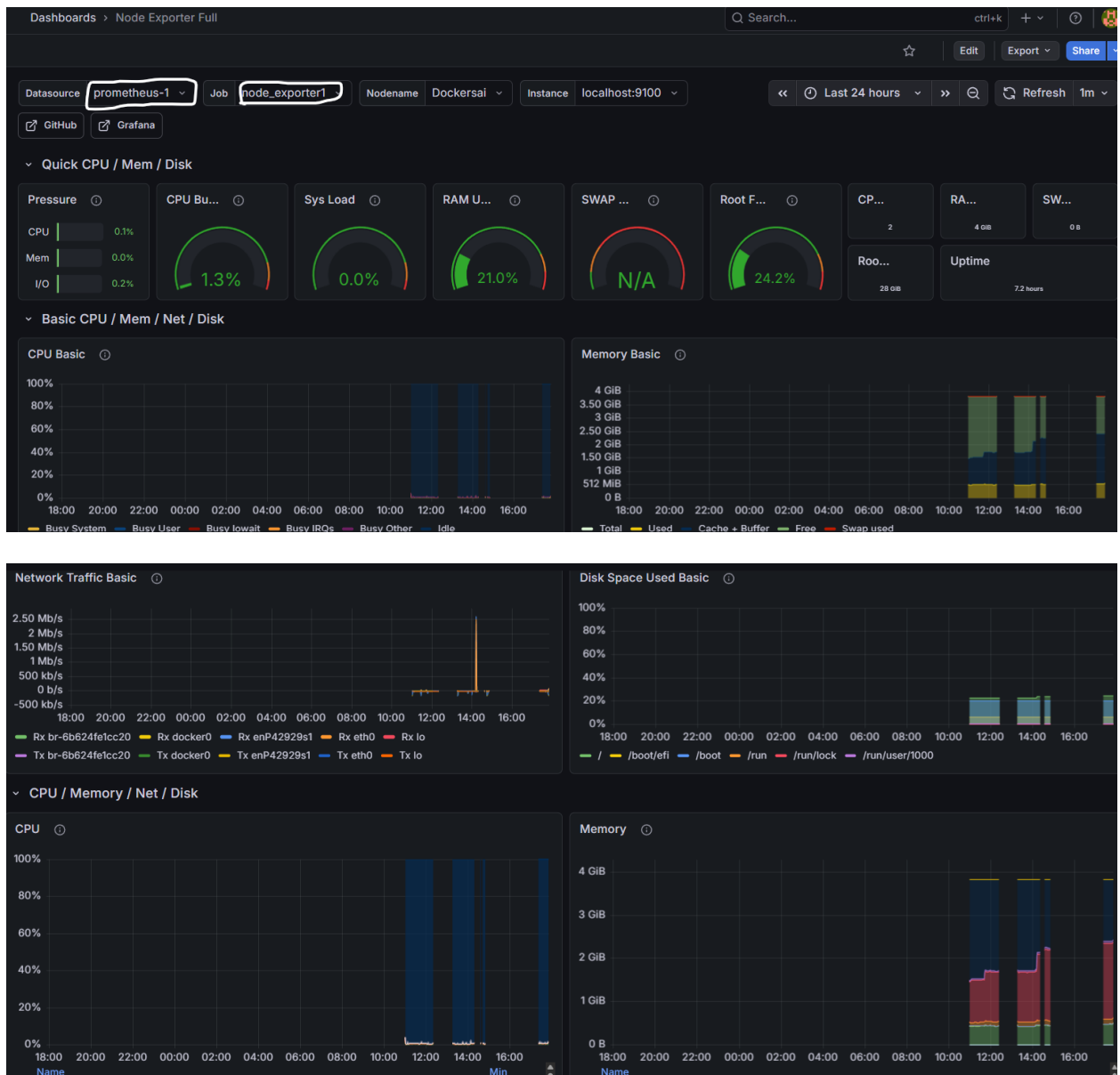
|

Import

Cancel

At ds_victoriametrics : Select prometheus 1 and import

Change data source and job to check it



Under administration -> general -> settings
Under connections -> add new connection ->
Top right you can find add new data source

Find this block :

Connection : Prometheus server URL * : <http://20.63.66.138:9090>

Enter your prometheus address : <ipaddress>:9090

Click on save and test

